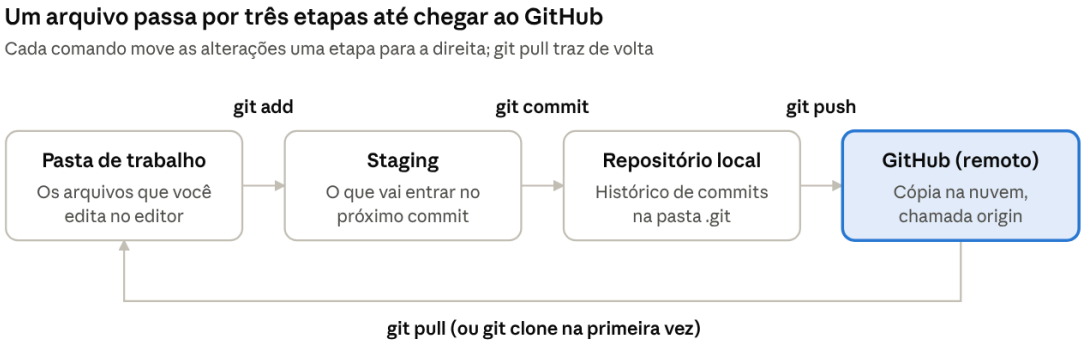
Tutorial 2 — Criar um repositório e comandos básicos do Git

Este tutorial parte do ponto em que o Tutorial 1 terminou: Git Bash instalado, conta no GitHub criada e chave SSH funcionando. Ao final você terá um repositório publicado no GitHub e saberá os comandos do dia a dia. As imagens de tela são ilustrações simplificadas.

Como o Git funciona



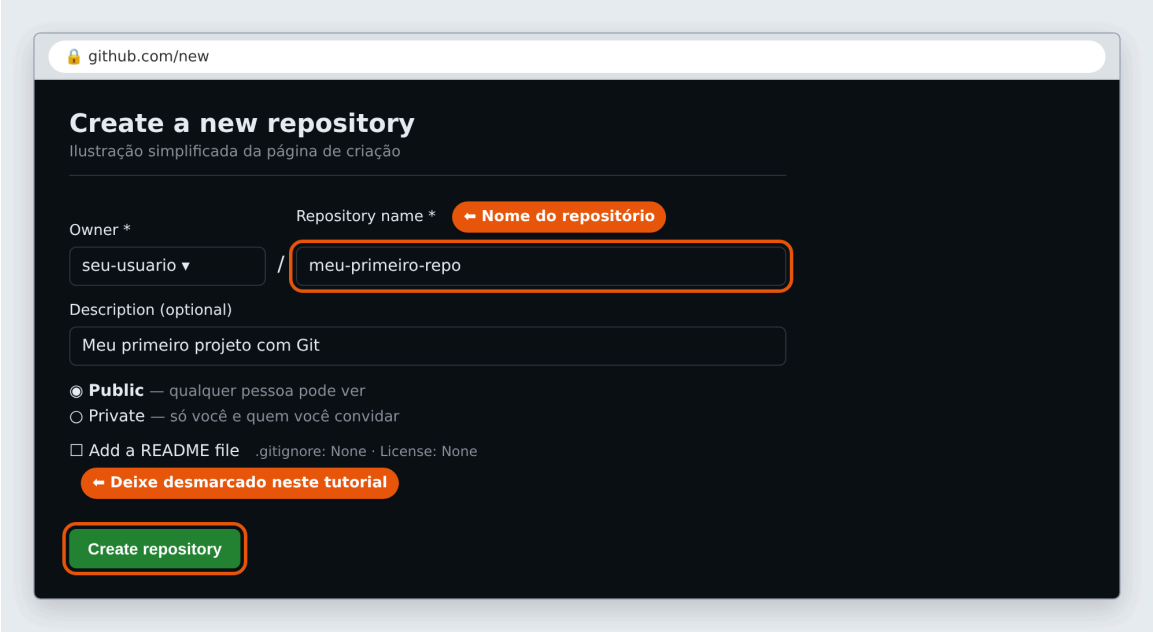
As quatro áreas do Git e os comandos que ligam cada uma

Você edita na pasta de trabalho, escolhe o que salvar com `git add`, grava com `git commit` e publica com `git push`. Enquanto você não der `git push`, os commits existem só no seu computador.

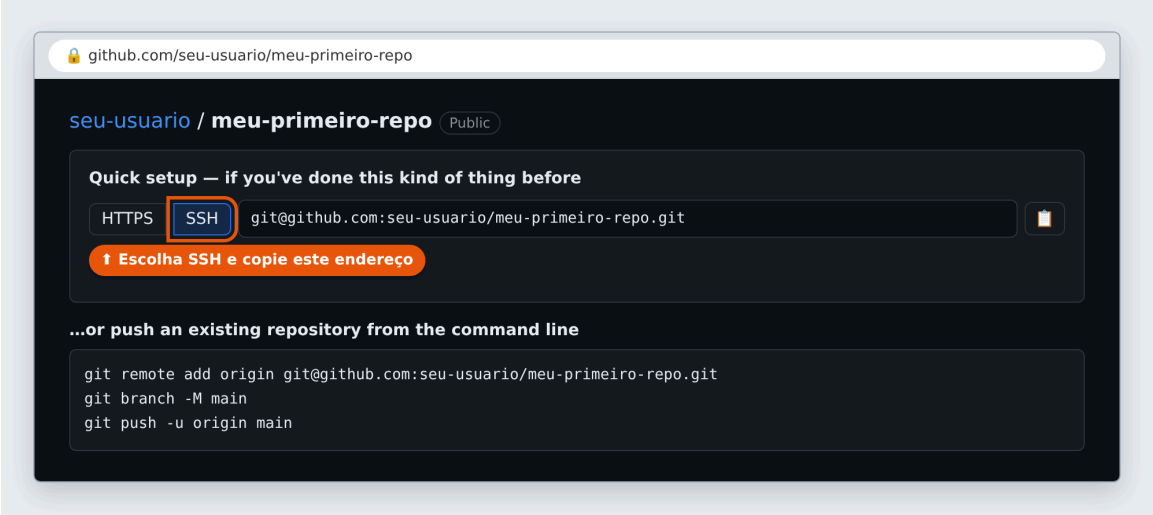
Parte 1 — Criar o repositório no GitHub

Um repositório é a pasta do projeto com todo o seu histórico. Vamos criar a versão online primeiro, vazia, e depois ligá-la a uma pasta no seu computador.

1. No GitHub, clique no **+** no canto superior direito → **New repository** (ou acesse github.com/new).
2. **Repository name:** `meu-primeiro-repo`. Use letras minúsculas, números e hífens, sem espaços nem acentos.
3. **Description:** opcional, uma frase sobre o projeto.
4. **Public** (qualquer pessoa vê) ou **Private** (só você e convidados).
5. **Deixe desmarcados** "Add a README file", ".gitignore" e "License". Um repositório vazio evita conflito no primeiro envio da Parte 2.
6. Clique em **Create repository**.



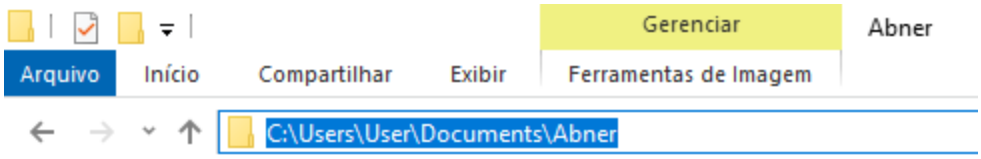
O GitHub abre a página *Quick setup*. Clique em **SSH** e copie o endereço, que tem o formato `git@github.com:seu-usuario/meu-primeiro-repo.git`. Você vai usá-lo na Parte 2.



Parte 2 — Criar o repositório local e enviar ao GitHub

2.1 Criar a pasta e iniciar o Git

Abra o Explorador de Arquivos e escolha o local de preferência para criar seu primeiro repositório, para então copiar o caminho da pasta selecionada e utilizar no terminal.



No Git Bash:

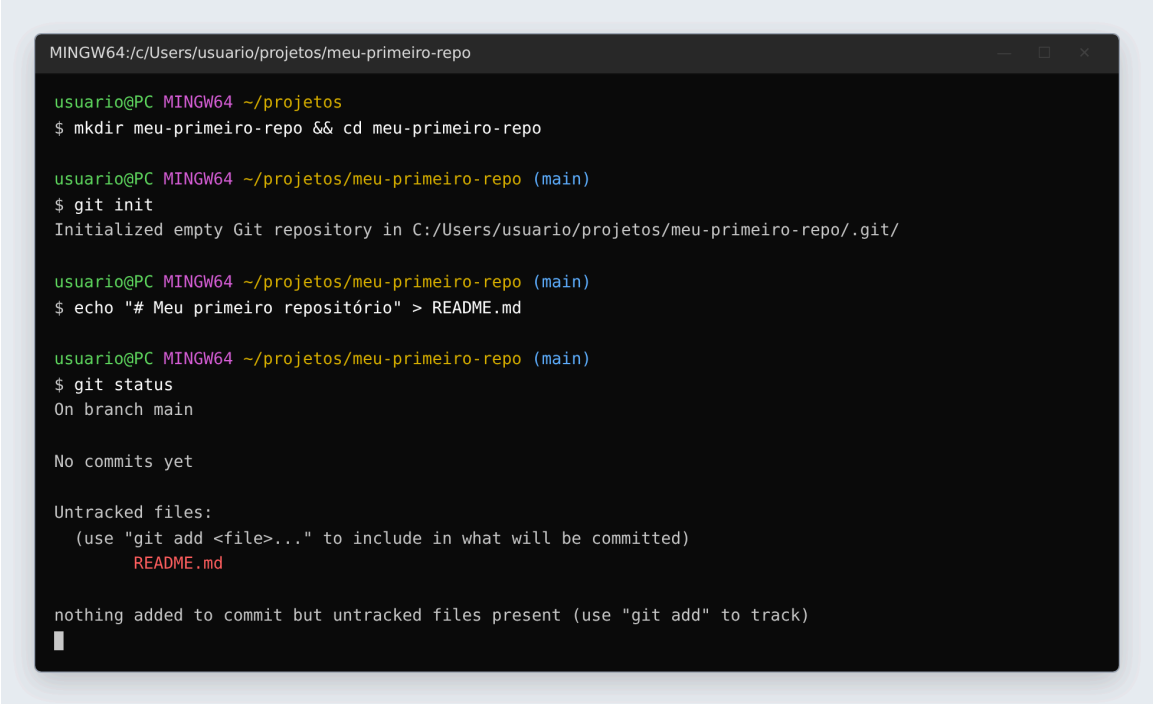
```
cd C:\Users\User\Documents\Abner
mkdir -p projetos && cd projetos
mkdir meu-primeiro-repo && cd meu-primeiro-repo
git init
```

`git init` cria a pasta oculta `.git`, onde fica todo o histórico. A partir daí o terminal mostra `(main)` ao lado do caminho: é o branch atual.

2.2 Criar um arquivo e ver o status

```
echo "# Meu primeiro repositório" > README.md
git status
```

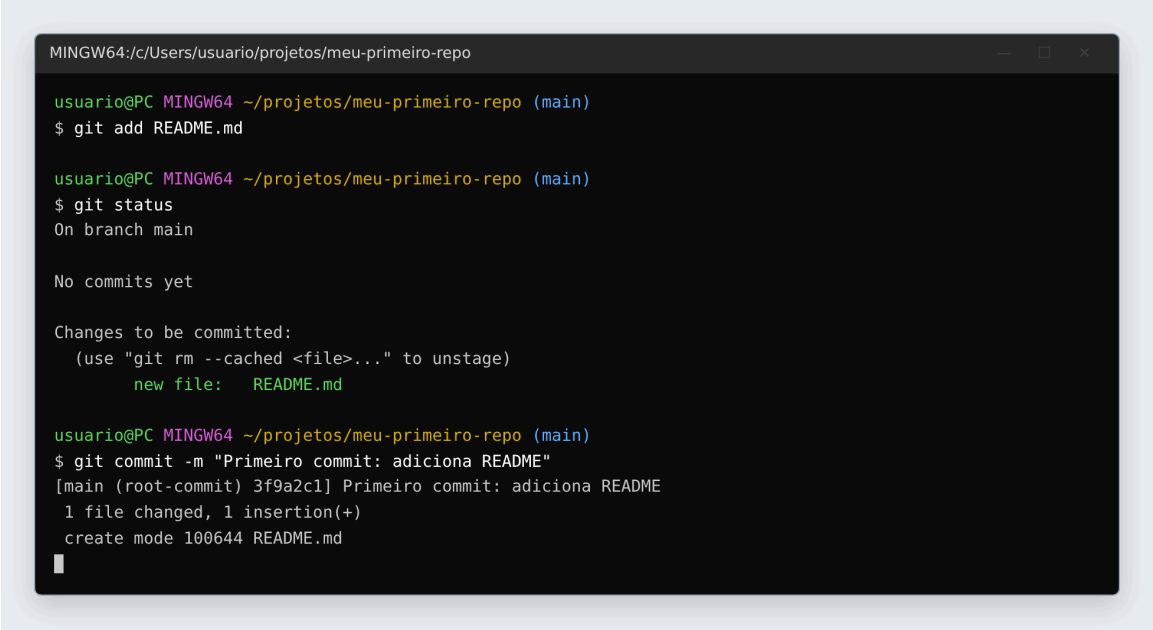
O `git status` mostra o arquivo em vermelho em *Untracked files*: o Git vê o arquivo, mas ainda não o acompanha.



2.3 Preparar (add) e salvar (commit)

```
git add README.md
git commit -m "Primeiro commit: adiciona README"
```

1. `git add` coloca o arquivo na **área de preparação (staging)**. No `git status` ele passa a aparecer em verde.
2. `git commit` grava uma "foto" dos arquivos preparados no histórico, com a mensagem após `-m`. Escreva mensagens curtas que digam **o que** mudou.

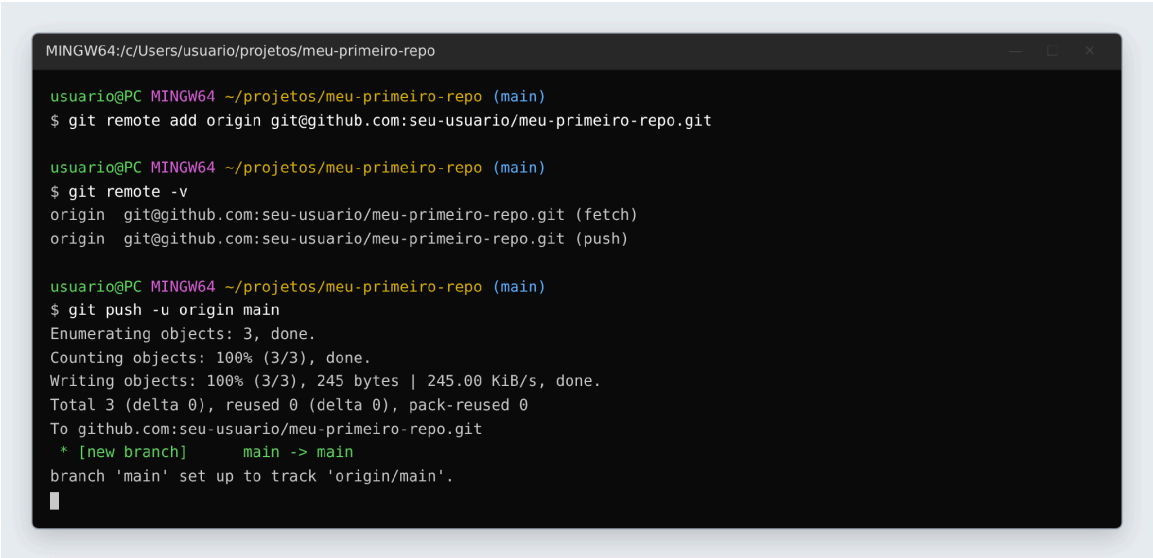


2.4 Ligar ao GitHub e enviar (push)

Cole o endereço SSH copiado na Parte 1:

```
git remote add origin git@github.com:seu-usuario/meu-primeiro-repo.git
git remote -v
git push -u origin main
```

1. `git remote add origin` dá o apelido **origin** ao endereço do GitHub.
2. `git remote -v` confere se o endereço está certo.
3. `git push -u origin main` envia o branch `main`. O `-u` memoriza o destino: daqui para frente basta `git push`.



Atualize a página do repositório no GitHub: o `README.md` aparece lá.

Parte 3 — Alternativa: começar pelo git clone

Se o repositório já existe no GitHub (seu, com README, ou de outra pessoa), não use `git init`. Baixe uma cópia completa com `git clone`:

```
cd C:/Users/User/Documents/Abner/projetos
git clone git@github.com:seu-usuario/outro-repo.git
cd outro-repo
```

O clone já vem com o histórico inteiro e com o `origin` configurado, então `git push` e `git pull` funcionam direto.

Parte 4 — O ciclo do dia a dia

Depois do primeiro envio, o trabalho segue sempre o mesmo ciclo:

1. **Atualize** antes de começar, para trazer mudanças feitas em outro computador ou por colegas:

```
git pull
```

2. **Edite** os arquivos no seu editor (VS Code, Bloco de Notas etc.).
3. **Veja o que mudou:**

```
git status        # quais arquivos mudaram
git diff          # o que mudou dentro deles, linha por linha
```

4. **Prepare e salve:**

```
git add .                          # o ponto adiciona todos os arquivos alterados
git commit -m "Descreve a mudança"
```

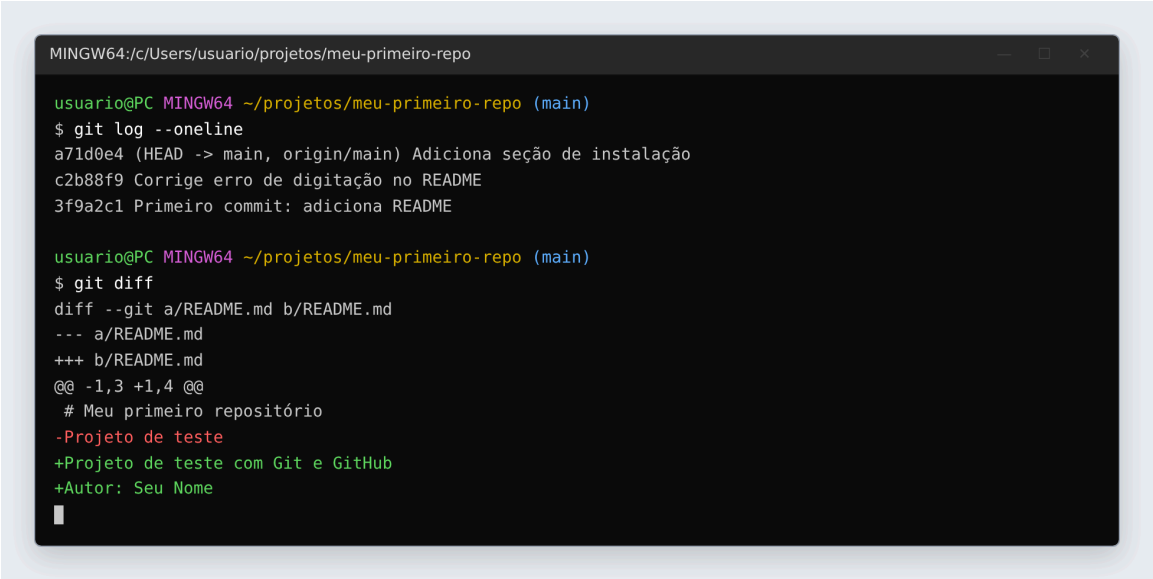
5. **Envie:**

```
git push
```

Ver o histórico

```
git log --oneline     # um commit por linha, o mais recente em cima
git log               # versão completa, com autor e data (aperte q para sair)
```

No `git diff`, linhas em vermelho com `-` foram removidas e linhas em verde com `+` foram adicionadas.



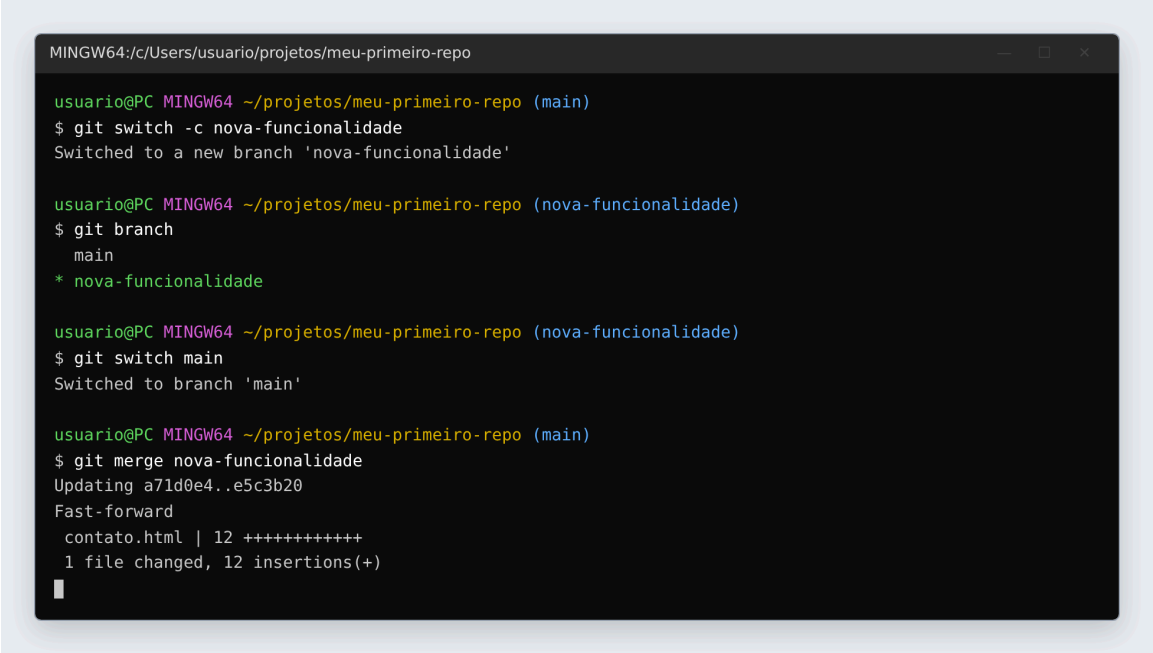
Parte 5 — Branches e merge

Um branch é uma linha paralela de trabalho. Você testa uma ideia sem mexer no `main` e só junta quando estiver pronta.

```
git switch -c nova-funcionalidade   # cria o branch e entra nele
# ... edite, git add, git commit ...
git push -u origin nova-funcionalidade   # opcional: envia o branch ao GitHub

git switch main                     # volta para o main
git merge nova-funcionalidade       # traz as mudanças para o main
git push

git branch                          # lista os branches (* = atual)
git branch -d nova-funcionalidade   # apaga o branch já juntado
```



> *Se o mesmo trecho de um arquivo foi alterado nos dois branches, o merge para com **CONFLICT**. Abra o arquivo, escolha o conteúdo certo entre as marcas `<<<<<<<`, `=======` e `>>>>>>>`, apague as marcas e finalize com `git add` e `git commit`.*

No GitHub, o caminho mais comum em equipe é enviar o branch e abrir um **Pull Request**, para que outra pessoa revise antes do merge.

Referência rápida

Comando	Para que serve
`git init`	Transforma a pasta atual em repositório Git
`git clone <endereço>`	Baixa um repositório existente
`git status`	Mostra arquivos novos, alterados e preparados
`git add <arquivo>` / `git add .`	Prepara um arquivo / todos para o commit
`git commit -m "mensagem"`	Grava os arquivos preparados no histórico
`git log --oneline`	Lista os commits, um por linha
`git diff`	Mostra o que mudou e ainda não foi preparado
`git remote add origin <endereço>`	Liga o repositório local ao GitHub
`git push`	Envia seus commits ao GitHub
`git pull`	Traz e junta os commits do GitHub

Comando	Para que serve
`git switch -c <nome>`	Cria um branch e entra nele
`git switch <nome>`	Troca de branch
`git merge <nome>`	Junta outro branch ao atual

Ignorar arquivos com .gitignore

Crie um arquivo chamado `.gitignore` na raiz do projeto e liste o que o Git não deve acompanhar, um item por linha:

```
node_modules/
.env
*.log
```

Nunca envie senhas, chaves ou o arquivo `.env` ao GitHub, mesmo em repositório privado.

Desfazer erros comuns

Situação	Comando
Descartar alterações de um arquivo ainda não preparado	`git restore arquivo.txt`
Tirar um arquivo da preparação (desfazer o `git add`)	`git restore --staged arquivo.txt`
Corrigir a mensagem do último commit, antes do push	`git commit --amend -m "nova mensagem"`
Desfazer um commit já enviado, criando um commit inverso	`git revert <código do commit>`

> *`git restore` descarta suas alterações sem volta. Confira com `git diff` antes.*

Fontes

- Git — documentação oficial (git-scm.com)
- GitHub Docs — Creating an account on GitHub

Versão para Linux

Todos os comandos Git deste tutorial são **idênticos** no Linux. A criação do repositório no GitHub (Parte 1) também não muda. Muda só o ambiente ao redor: você usa o terminal do sistema (**Ctrl+Alt+T** no Ubuntu) em vez do Git Bash, e os caminhos começam em `/home/usuario`.

Pré-requisito: a seção **Versão para Linux** do Tutorial 1 concluída (Git instalado, `init.defaultBranch main` configurado e chave SSH no GitHub).

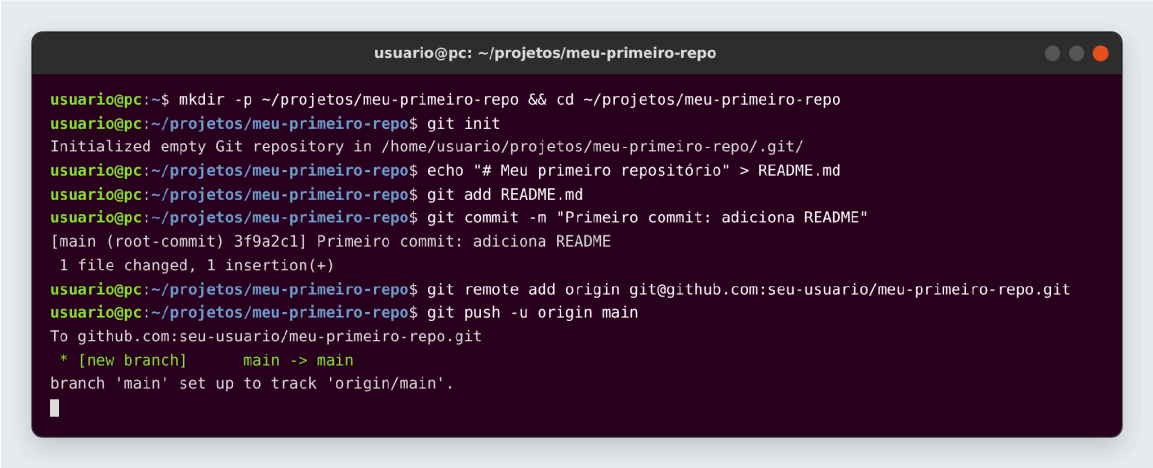
Passo a passo completo no terminal do Linux

```
# 1. Criar a pasta e iniciar o repositório
mkdir -p ~/projetos/meu-primeiro-repo
cd ~/projetos/meu-primeiro-repo
git init

# 2. Primeiro arquivo e primeiro commit
echo "# Meu primeiro repositório" > README.md
git status
git add README.md
git commit -m "Primeiro commit: adiciona README"

# 3. Ligar ao GitHub e enviar
git remote add origin git@github.com:seu-usuario/meu-primeiro-repo.git
git push -u origin main

# 4. Dia a dia
git pull
git status
git add .
git commit -m "Descreve a mudança"
git push
```



O que muda no Linux

Situação	No Linux
Branch não aparece no prompt, como `(main)` no Git Bash	Normal no Ubuntu. Use `git status` ou `git branch` para ver o branch atual
Colar comandos no terminal	**Ctrl+Shift+V** (Ctrl+V não funciona no terminal)
Abrir a pasta no gerenciador de arquivos	`xdg-open .`
Editar um arquivo pelo terminal	`nano README.md` (salve com Ctrl+O, saia com Ctrl+X)
`git log` ou `git diff` abriram uma tela que não fecha	Aperte **q** para sair (igual ao Windows)

Situação	No Linux
Arquivos que começam com ponto (`.gitignore`, `.git`) não aparecem	São ocultos: use `ls -a` ou **Ctrl+H** no gerenciador de arquivos
Projeto compartilhado com quem usa Windows	Rode uma vez `git config --global core.autocrlf input` para evitar diferenças de fim de linha

> *Para ver o branch no prompt como no Git Bash, adicione ao final do arquivo `~/.bashrc` a linha `PS1='\u@\h:\w$(__git_ps1 " (%s)")\$ '` e abra um novo terminal. Funciona no Ubuntu e no Debian, que já carregam a função `__git_ps1`.*